ES - Quality Assurance Manual

João Roldão - 113920

Hugo Castro - 113889

Hugo Sousa - 112733

Henrique Teixeira - 114588

João Varela - 113780

41492 - Engenharia de Software

deti universidade de aveiro departamento de eletrónica, telecomunicações e informática

Table of Contents

1. Project Management **3**
1.1 Assigned Roles 3
1.2 Backlog Grooming and Monitoring 3
2. Code Quality Management **4**
2.1 Coding Standards & Practices 4
2.2 Code Review Policy 4
3. Continuous Delivery Pipeline (CI/CD) **5**
3.1 Development Workflow 5
3.2 Continuous Integration (CI) 5
3.3 Observability and Monitoring 6
4. Software Testing **6**
4.1 Overall Testing Strategy 6
4.2 Functional Testing 7
4.3 Developer-Facing Tests 7
4.4 Exploratory Testing 8
4.5 Non-Functional Testing 8

1. Project Management

1.1 Assigned Roles

Role	Name
Scrum Master	João Roldão
Product Owner	Hugo Castro
QA Engineer	Hugo Sousa
Service Analyst	Henrique Teixeira
DevOps Master	João Varela

1.2 Backlog Grooming and Monitoring

- **Backlog management:** The team uses **GitHub Projects and Issues** to manage tasks.
- **Work organization:**
  - Issues represent individual tasks, bugfixes, or features.
  - Larger features can be grouped into epics (project-level categories).
- **Backlog refinement:** Occurs at least weekly, in the beginning of each sprint, where issues are reviewed, clarified, and prioritized by the Product Owner with team input.
- **Progress tracking:**
  - GitHub Kanban board is used to visualize tasks in columns (Backlog, To Do, In Progress, Review, Done).
  - Labels and milestones are applied for categorization and deadlines.
- **Traceability:** Each PR must be linked to its corresponding GitHub issue, ensuring visibility from requirement to delivery.
- **Review cadence:** Progress is reviewed in weekly team meetings and retrospectives.

2. Code Quality Management

2.1 Coding Standards & Practices

- **Style guides:** Adopt language-specific conventions (e.g., Java → Google Java Style; TypeScript/JS → ESLint + Prettier).
- **Static analysis (on Pull Requests):** Linting and formatting are mandatory before pushing (e.g., via husky/lint-staged).
- **Naming & structure:** Use clear, descriptive names; keep functions short with single responsibility; comments only when the code is not self-explanatory.
- **Code documentation:** Javadoc/TSdoc for public APIs; module-level README files with build/test instructions.

2.2 Code Review Policy

- **Branch protection:** Direct commits to *develop* and *main* are **strictly forbidden**.
- **Pull Requests required:** All code changes must go through a PR.
- **Minimum approval:** At least **1 reviewer** is required (ideally someone other than the author).

  Mandatory checks before merge:
  - CI builds are successful.
  - Linting/formatting checks passed.
  - Unit/integration tests passing.
- **PR requirements:** descriptive title, linked issue/task, and testing evidence (logs, screenshots, coverage).
- **Review checklist:** functional correctness, readability, performance implications, security, tests, documentation updates.
- **PR size:** Prefer small, focused PRs; large changes must be split into multiple PRs.
- **Author's responsibility:** respond to feedback, update the PR accordingly, and maintain a clean history (squash/rebase as needed).

3. Continuous Delivery Pipeline (CI/CD)

3.1 Development Workflow

Branching model:

- The team follows a **GitHub-based workflow** with *develop* as the integration branch and *master* as the production branch.
- **Feature development:** each new feature, bugfix, or improvement is first created as a **GitHub issue** in the project backlog.
- **Naming convention for branches:**
    - feat/<short-description>
    - setup/<short-description>
    - docs/<short-description>
    - task/<short-description>
    - bug-fix/<short-description>
- Commits should be clear and descriptive, providing enough context to understand what changes were made.
- Once complete, the branch is pushed and a **Pull Request (PR)** is opened to *develop*.
- After review and approval, changes are merged into *develop*.
- Releases are promoted by merging *develop* into *main*, which represents production-ready code.

Definition of Done for a feature/PR:

- Code implemented and pushed.
- The linked GitHub issue moved to “In Review” or “Done”.
- Code reviewed and approved by at least one teammate.
- CI pipeline (build + tests + static analysis) passes.
- Documentation updated if applicable.

3.2 Continuous Integration (CI)

- **Tooling:** GitHub Actions is used as the CI engine.
- **Triggers:**
    - On every PR to develop or master/main.
    - On push events to feature branches.
- **Checks performed:**
    - Build compilation and dependency installation.
    - Linting and formatting checks.
    - Unit and integration tests executed.
- **Outcome:**
    - A PR cannot be merged if any CI check fails.
    - Test results and code quality reports are published as part of the PR.

3.3 Observability and Monitoring

- **Tooling:** GitHub Actions is used for automated deployment workflows.
- **Containerization:**
    - Applications are packaged into **Docker images**, ensuring consistency across environments.
- **Deployment process:**
    - On merge/push to master/main, a CD workflow is triggered.
    - The workflow builds and pushes the Docker image to the container registry.
    - The image is automatically deployed to the target environment
- **Automation benefits:**
    - Minimal manual intervention; deployments are fast, repeatable, and consistent.
    - Reduced risk of human error.
    - Immediate availability of new features and fixes in the production environment.

4. Software Testing

4.1 Overall Testing Strategy

The project adopts a **layered testing approach** to ensure quality across all levels of the system.
Key principles:

- **Test early, test often:** defects should be caught as soon as possible.
- **Automation first:** automated tests are preferred whenever feasible.
- **Shift-left testing:** developers are responsible for writing unit and integration tests alongside implementation.

Test execution is integrated into the CI pipeline, making successful test results a requirement for merging code. A minimum of 70% test coverage is required to maintain code quality and ensure sufficient validation of system functionality.

Primary tools and frameworks:

- **JUnit 5** – core framework for unit and integration tests.
- **Mockito** – mocking framework for isolating dependencies.
- **Cucumber** – for acceptance test–driven development (ATDD) and behavior-driven development (BDD).
- **Selenium** – for end-to-end (E2E) and UI automation.
- **Grafana** – for monitoring and analyzing test metrics and system performance.

4.2 Functional Testing

- **Black-box perspective:** verifies system behavior against business requirements without reference to internal implementation.
- **Scope:** implemented for all major features, workflows, and user-facing functionality.
- **Implementation:**
    - Written using **Cucumber** for readability and collaboration.
    - Derived directly from acceptance criteria in GitHub issues.
- **Regression suite:** functional tests are maintained as part of a growing regression suite, run automatically in CI.

4.3 Developer-Facing Tests

- **Unit Tests:**
    - Mandatory for all business-critical logic, validation, and utility functions.
    - Written by the developer implementing the feature.
    - **JUnit 5** for test execution.
    - **Mockito** for mocking services, repositories, and external dependencies.
- **Integration Tests:**
    - Required when multiple layers (controllers, services, repositories) or external APIs interact.
    - Database and API validations run with **JUnit 5 + TestContainers**.
- **Execution:**
    - Both unit and integration tests run automatically on every PR via GitHub Actions.

4.4 Exploratory Testing

- Complements scripted test cases, focusing on discovering issues through unscripted investigation.
- Conducted by developers and QA engineers during sprint reviews or before major releases.
- Focus areas:
    - New or complex user flows.
    - Features with multiple input combinations.
    - Areas affected by recent changes or regressions.
- Outcomes: findings are documented in GitHub Issues as bugs or improvement suggestions.

4.5 Non-Functional Testing

Non-functional testing ensures that the system not only works functionally but also meets defined **performance,availability, reliability and maintenance expectations**. Testing in this area validates **how** the system behaves under different conditions.

- **Service Level Objectives (SLOs):**
  - API requests should complete within **3 seconds under normal load**.
  - Order registrations should be processed within **2 seconds**.
  - Consistency across modules maintained; carrier API failures handled with retries.
- **Performance testing:**
  - Load and stress testing using **K6**.
  - Results visualized and monitored through **Grafana dashboards**.
  - Focus on throughput, concurrency, and resource usage (CPU, memory, network).
- **Execution:**
  - Non-functional tests are not part of every CI run.
  - Executed before major releases or as part of scheduled test cycles.
  - Reviewed jointly by QA and DevOps.